Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Derrick Choi
@derrickcchoi

Article

Codex ran uninterrupted for 25 hours using GPT-5.3-Codex and built a design tool

26 61 725 106K

I work on the #Codex team at @OpenAI and I wanted to see what a long-running AI teammate looks like without needing to baby the agent.

Since I have the luxury of being able to use and test GPT-5.3-Codex internally before launch, I decided to run Codex at “Extra High” reasoning and full access mode and asked it to implement a design tool from scratch. It ended up running for ~25 hours uninterrupted, using ~13M total tokens, and generating ~30k LOC.

Reminder that this was more of an experiment, not an enterprise grade rollout, and I wouldn’t ship the initial output without further review. But Codex did really well overall on the parts that matter for a long-run agent: it followed the spec, reasoned on what it needed to do, passing verification steps, and the result exceeded my expectations.

If you're interested in diving deeper yourself, I have the code on GitHub in a public repo (design-desk). Built with React, TypeScript, Vite, and Tailwind.

I asked Codex to generate a simple summary page of the session data as well.

Search
Relevant peo
Derrick Cho
@derrickcch
Codex @Op
Ops at @Sca
SWE @Appl
What’s happe
Get State Farm
Stop livin’ on a pray
Promoted by State
Trending in United Sta
Herbie
Trending in United Sta
Zach Bryan
Trending
#OpenClaw
Show more
Terms of Service | Priva
Accessibility | Ads info

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

High level summary of session

Session data in the CLI

1. Why long-running agents are becoming more real

This isn't just "models got smarter." The real shift is time horizon: agents can now stay coherent longer and complete bigger chunks of work end-to-end. In November 2025, we launched GPT-5.1-Codex-Max, which was the first model we trained to compact across multiple context windows.

METR's time-horizon evaluations are a good framing: the length of software tasks frontier agents can complete with ~50% and 80% reliability has been climbing fast, with a rough ~7 month doubling time. And OpenAI models currently sit at the top in both categories.

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Our recent GPT-5.3-Codex launch (announcement) pushes this further for agent work in two practical ways:

1. It's better at multi-step execution (plan → implement → validate → repair).
2. It's easier to steer mid-flight without resetting the whole run (course corrections don't wipe progress).

I was also inspired by Cursor's recent blog post (https://cursor.com/blog/scaling-agents) where they built a browser by running GPT-5.2-Codex uninterrupted for a week which resulted in 3M lines of code written.

The Cursor team wrote that OpenAI models are

> much better at extended autonomous work: following instructions, keeping focus, avoiding drift, and implementing things precisely and completely.

2. The harness matters as much as the model

Long-running work is less about one giant prompt and more about the agent loop the model operates inside.

In Codex, the loop looks like: plan → edit → run tools (tests/build/lint) → observe results → repair → document → repeat. That gives the agent

- Real feedback (errors, diffs, logs)
- Keeps state externalized (repo + worktrees + docs)
- Makes long runs steerable: you can correct direction based on outcomes

This is also why Codex models feel better on Codex surfaces than a generic chat window: the harness supplies structured context (repo metadata, file tree, diffs, command outputs) and enforces a disciplined "done when" routine.

We recently published an article about the Codex agent loop that has more details.

To top this off, we also launched the Codex app this week makes that loop usable day-to-day:

- Parallel threads across projects (long work doesn't block your day job)
- Skills (standardize plan/implement/test/report)
- Automations (routine work in the background)
- Git worktrees (isolate runs, keep diffs reviewable, reduce thrash)

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

3. What I did (the setup)

I picked a design tool for this “experiment” because it’s an unforgiving test: UI + data model + editing operations + lots of edge cases. You can’t bluff it. If the architecture is wrong, it breaks quickly.

I gave GPT-5.3-Codex a meaty spec, ran it at “Extra High” reasoning, and it ended up running uninterrupted for ~25 hours and was able to stay coherent and ship quality code. The model also ran verification steps (tests, lint, typecheck) for every milestone it completed.

The key idea was durable project memory. I wrote down the spec, plan, and constraints in markdown files that Codex could repeatedly reference. That prevented drift and kept a stable definition of “done.”

File stack (linked to each file on GitHub):

- Prompt.md (spec + deliverables)
- Plans.md (milestones + validations)
- Architecture.md (principles + constraints)
- Implement.md (execution instructions referencing the plan)
- Documentation.md (status + decisions as it shipped)

Codex doesn't just write code and hope it works. After each milestone, it runs the repository’s verification commands such as build/tests/lint. If something fails, like npm run lint, it stops, fixes the issue, and only then moves on to the next milestone (example below).

Npm run commands that Codex ran

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Codex correcting issues after npm run lint

The output is by no means perfect or something I'd call fully production ready, but real enough that "it compiles" isn't the bar. Like I mentioned above, the bar was: does the application adhere to my instructions and does it actually work?

4. The "memory stack"

Prompt.md (kickoff contract)

Purpose: Freeze the target so the agent doesn't "build something impressive but wrong."

Key sections in the file:

- Goals + non-goals
- Hard constraints (perf, determinism, UX, platform)
- Deliverables (what must exist when finished)
- "Done when" (checks + demo flow)

As you can see from the screenshot below, this generated the Plans markdown file.

The prompt I gave Codex to kickstart the work

Plans.md (milestones + validation harness)

Purpose: Turn open-ended work into a sequence of checkpoints the agent can finish and verify.

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Key sections in the file:

- Milestones small enough to complete in one loop
- Acceptance criteria + validation commands per milestone
- Stop-and-fix rule: if validation fails, repair before moving on
- Decision notes to avoid oscillation

Codex referring to the Plans markdown file as it works

Architecture.md (principles + constraints)

Purpose: Long runs fail when the agent “invents new architecture every hour.” This file prevents that by freezing the invariants.

Key sections in the file:

- Guiding principles (what patterns are allowed)
- Data model + boundaries (where state lives, what’s pure, what’s UI)
- Determinism rules (IDs, serialization, ordering, replay)
- Collaboration/sync assumptions (if relevant)

Implement.md (execution prompt)

Purpose: This is the runbook. It tells Codex exactly how to operate: follow the plan, keep diffs scoped, run validations, update docs.

Key sections in the file:

- Plans markdown file is source of truth (milestone-by-milestone)
- Run validation after each milestone (fix failures immediately)
- Keep diffs scoped (don’t expand scope)
- Update documentation markdown file continuously

Here is a screenshot of the summarized session data where Codex continuously compacted context after I gave the go-ahead to

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

implement.

My prompt for Codex to read the Implementation markdown file

Documentation.md (living status + decisions)

Purpose: This is the shared memory and audit log. It's how I can step away for hours and still understand what happened.

What matters inside:

- Current milestone status (what's done, what's next)
- Decisions made (and why)
- How to run + demo (commands + quick smoke tests)
- Known issues / follow-ups

Codex updating high level milestone status as it works

Commands that Codex runs to verify quality

5. What Codex actually built

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Codex didn't just scaffold a repository that compiles. It produced a working application with real interaction surfaces and stateful features you can click through end-to-end: a canvas editor, precision inspector controls, layers management, alignment/snapping, collaboration primitives, history + replay, prototype preview, comments, and export.

What was implemented (high level):

1. Canvas editing: frames, groups, shapes, text, images/icons, buttons, charts
2. Live collaboration: Presence, cursors, selections, and edits sync across tabs in the same session
3. Inspector: geometry, styling, and text controls
4. Layers: search, rename, lock/hide, reorder via drag + drop
5. Guides: grid/align/distribute snapping during drag
6. History: version snapshots + restore
7. Replay: time-travel edit timeline + branch from any point
8. Prototype mode: hotspots + flow navigation
9. Comments: pinned threads with resolve/reopen
10. Export: save/import/export + CLI export to JSON and React + Tailwind

A few screenshots below:

Live collaboration with a teammate

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

View and restore snapshots of project

Time travel around the edit timeline

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Ability to comment on the canvas

Is everything perfect? Definitely not. I can always spend a lot more time prompting Codex, adding features, and ironing out minor bugs. But the fact that Codex was able to one-shot this with pretty minimal guidance from me is truly impressive.

As I mentioned above, you can run the app locally on your computer by going to the design-desk repository where you can see all the details. Feel free to play around with it yourself!

6. Conclusion

This 25 hour long Codex run is a preview of where “building with code” is going: software work shifts from single-shot prompts and pair-programming loops to long-running teammates that can take a real slice of a project end-to-end, with you steering at milestones instead of micromanaging every line.

Our direction with Codex is clear: more capable teammate behavior, developers turn into accelerated experts when the agent absorbs routine implementation and verification work, freeing humans up for the parts that still matter most: design, architecture, product decisions, and the novel problems that don’t have a template.

Want to publish your own Article?
Upgrade to Premium

10:02 AM · Feb 5, 2026 · 106.2K Views
86 1.1K 785 1.1K
Relevant View quotes
Post your reply Reply

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr

Home
Explore
Notifications
Follow
Chat
Grok
Bookmarks
Premium
Profile
More
Post

Jonathan Bailey
@jnnyslshr